

二、研究計畫內容：

(一) 摘要

在軟體專案維護過程中，錯誤追蹤系統(Issue Tracking System)是管理錯誤狀態的核心工具。當使用者回報專案的異常時，此系統會生成對應的 Ticket，作為工程師分析與修復問題的依據。本專題旨在開發並實作一套「基於 LLM 的軟體錯誤追蹤與修復系統」，預計運用 LLM 判讀使用者所提交 Ticket 內之錯誤描述與執行環境等資訊，自動判定處理該 Ticket 的負責人，並按照提報的訊息生成建議的補丁，供負責人參考。過程中，此系統將自動化判定 Ticket 的優先級(包含：Blocker、Critical、Major、Minor、Trivial，如表一說明)、推測程式碼錯誤的位置(後續簡稱錯誤定位)，並藉此自動生成程式碼修補補丁與對應的測試案例，且草擬需提交的程式碼異動訊息(Commit Message，後續簡稱提交訊息)。換句話說，此系統的目的是在使用者回報錯誤後促進整個處理流程的半自動化，期望能降低工程師在錯誤判定與修復的負擔，並提升開發效率與錯誤修復品質。

表一、Ticket 優先級說明(參考自 Trac¹預設分級)

優先級		定義
等級	名稱	
P1	Blocker(阻礙性)	系統無法運作或功能完全停擺，必須立刻修復，否則無法進行後續開發。
P2	Critical(關鍵)	嚴重影響功能，且無合理的替代方案。
P3	Major(主要)	功能有缺陷但有替代方案，或是功能缺失但未達 Critical 等級。是此系統對 Ticket 的初步預設等級。
P4	Minor(次要)	細微錯誤、介面問題或輕微功能缺陷，對整體系統運作影響較小。
P5	Trivial(輕微)	拼字錯誤、美觀問題或建議事項。

(二) 研究動機與研究問題

2.1 研究動機：

在軟體專案開發與維護過程中，錯誤追蹤系統長期扮演協助錯誤回報、問題追蹤與修復工作分派的重要角色，其運作流程如圖一。當使用者或測試人員發現專案所開發的系統執行異常時，通常會以 Ticket 形式提交錯誤描述、優先級與受影響的程式模組和版本等圖二所示資訊，以作為工程師後續分析與修復的依據。然而，在實務上，錯誤回報多以自然語言呈現，內容常包含模糊描述或截圖，且多數使用者所提供的資料缺乏完整的環境參數與 Log 等除錯的重要參考資訊，導致資訊完整度與描述方式差異甚大。另外，在傳統的錯誤追蹤系統中，工程師必須仰賴對系統架構、程式功能及歷史修改紀錄的熟悉程度，以純人工的方式解讀這些非結構化資訊後才可進行 Ticket 判讀和錯誤定位，並擬定修復策略。

以人工作業判讀 Ticket 依賴工程師的經驗與專業能力，隨著專案規模擴大與 Ticket 數量增加，會衍生出多項挑戰。首先，工程師在分析過程需投入大量時間理解與整理資訊，導致所需的人力與時間成本顯著提高，且初步分析的品質容易因工程師經驗差異而缺乏一致性，可能因而加重後續撰寫補丁與用於驗證的測試案例之負擔。其次，多數專案在長期運作下會累積大量歷史 Tickets，其中不乏內容相似或重複(Duplicate)回報的問題。若缺乏有效的輔助機制，工程師往往需憑經驗人工比對過往紀錄，以判斷是否為既有問題的重現或延伸。此類人工比對方式不僅耗時，亦容易遺漏重複問題，影響 Tickets 的分流判斷與資源配置的合理性。再者，若 Ticket 所描述的問題被判定為錯誤，開發人員在除錯過程必須先進行錯誤定位，再根據其設計補丁

¹ <https://trac.edgewall.org/>

以嘗試修復，並設計測試案例以驗證補丁的正確性。然而，若以人工進行錯誤定位和補丁生成與驗證，皆可能耗費大量時間。面對緊湊的開發時程與壓力，這些除錯工作容易耽誤工程的開發進度，也可能導致專案進度落後。



圖一、軟體錯誤回報與處理流程

圖二、錯誤追蹤系統創建 Tickets 的截圖(以 Trac 為例)

有鑑於 LLM 在程式碼理解與生成所展現的強大能力，本專題計畫以既有錯誤追蹤系統為基礎，運用 LLM 設計一套問題分析與修復的輔助系統。此系統將針對使用者回報的 Tickets 進行自動化整理與分析，除了提供結構化的問題分類結果，亦整合歷史紀錄比對功能，自動檢索過往 Tickets 以輔助判斷回報的內容是否重複，同時利用 LLM 進行自動化錯誤定位，協助開發者快速鎖定問題所在的程式碼區段。此外，我們將更進一步自動生成修復補丁及用於驗證補丁的測試案例供開發人員參考。最後，我們也預計實作自動生成提交訊息草稿的機制，提升系統開發與維護的便利性與效率。透過上述自動化輔助機制，期望能改善人工判定耗時與品質不穩的問題，讓工程師能將精力專注於審核決策和新功能的開發，進而提升整體軟體開發與維護的效率與品質。

2.2 研究問題：

本專題關注的重點並非如何取代工程師的決策，而是探討此類輔助機制在實際開發流程中的可行性與實用性。因此，在開發此系統的過程，我們將會探討並嘗試解決以下問題，以作開發此系統與評價所得成果的參考：

1. 非結構化資料處理：如何有效利用 LLM，將使用者所回報的模糊錯誤描述與 Log 等資訊轉化為結構化的 JSON 格式？
2. 歷史 Tickets 紀錄比對的準確性：透過比對歷史 Tickets 紀錄，系統是否能有效辨識重複回報的問題？
3. Ticket 判定準確性：LLM 能否準確判斷 Tickets 的優先級及負責人？

4. 透過系統所推測的錯誤所在位置，是否能協助工程師快速縮小程序碼除錯時的檢查範圍？
5. 運用 LLM 生成補丁及測試案例的可行性：
 - 5a. 所生成的程式碼補丁是否能正確修改使用者所描述的異常？
 - 5b. 系統針對所生成補丁而產生的測試案例是否具備足以重現使用者回報錯誤的驗證能力？
6. 生成的提交訊息草稿內容是否有效說明程式碼修改原因與範圍？

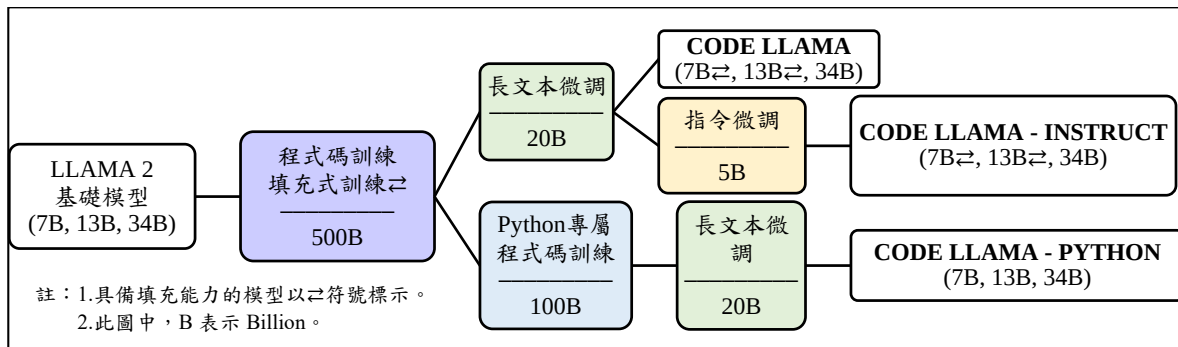
（三）文獻回顧與探討

3.1 透過 LLM 自動分析的階段

本專題的 LLM 自動分析工作流程始於資料正規化，擷取使用者輸入的模糊描述轉化為結構化 JSON 格式。隨後進入自動分類與分流階段，先比對以辨識重複 Ticket，若重複則此 Ticket 被標記為 Duplicate，若為新 Ticket 則預測其優先級(P1-P5)，再將 Ticket 指派給合適的負責人。在錯誤定位與建議生成階段，先定位錯誤的根源，再生成程式碼的修復補丁，也針對補丁產生重現錯誤的測試案例。後續章節 3.1.1 至 3.1.3 將針對前述工作回顧現有文獻的作法。

3.1.1 資訊擷取

據 Rozière 等人[1]的研究成果，Code Llama²系列模型分為三種主要變體，包含(1)基礎 Code Llama、(2)Python 專用，及(3)Instruct。針對指令執行需求，Code Llama-Instruct 進一步整合 Llama 2 的 RLHF V5 數據集與安全微調流程(流程如圖三所示)，顯著提升模型能夠準確地理解使用者意圖的能力，並提供相關且符合格式要求的回答，亦可有效防止生成虛假、具偏見或惡意的內容。

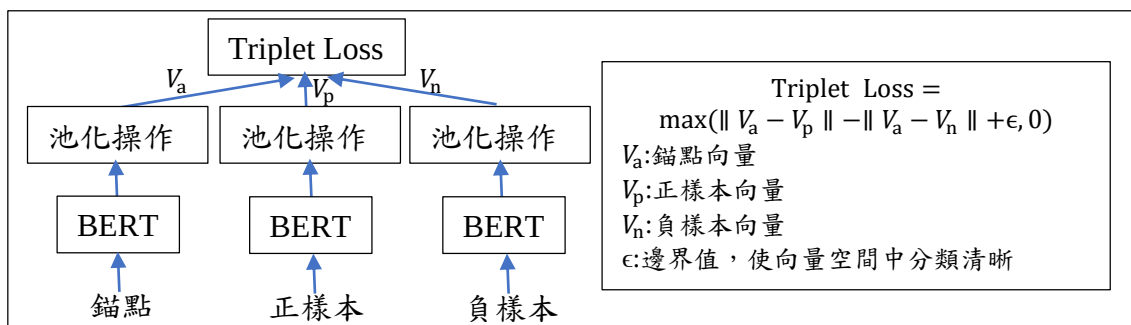


圖三、Code Llama 模型之訓練與微調流程[1]

3.1.2 Tickets 分類與分流

若耗費時間處理重複回報的 Tickets，會造成開發團隊資源的浪費。因此，要針對新增的 Ticket 比對歷史紀錄，判定是否存在已回報重複問題的 Tickets，此部分為輔助傳統流程圖一的步驟 1。傳統檢測方法如逆向檔案頻率方法 Term Frequency-Inverse Document Frequency [2]或 Latent Dirichlet Allocation [3]，但這些方法僅依賴詞彙統計，難以捕捉不同使用者所描述內容的深層語意關聯，其實驗平均精度均值往往偏低。針對此挑戰，Isotani 等人[4]採用改良自 BERT 模型的 Sentence-BERT(SBERT)技術[5]，透過在輸出層加入池化操作，將句子轉換為固定長度的語意向量。其研究成果證實，此技術大幅提升相似度檢索的運算效率。此外，該研究為了使模型適應軟體缺陷報告的特性，採用 Triplet Network(如圖四所示)針對標題與內容進行獨立向量化與微調，藉由同時輸入欲檢查句子(錨點)、與錨點相似的句子(正樣本)、與錨點不同的句子(負樣本)進行訓練，迫使模型拉近正樣本與錨點間的向量距離，並推遠負樣本。文獻[4]證實其檢測效能顯著優於傳統基準。

² <https://ollama.com/library/codellama>

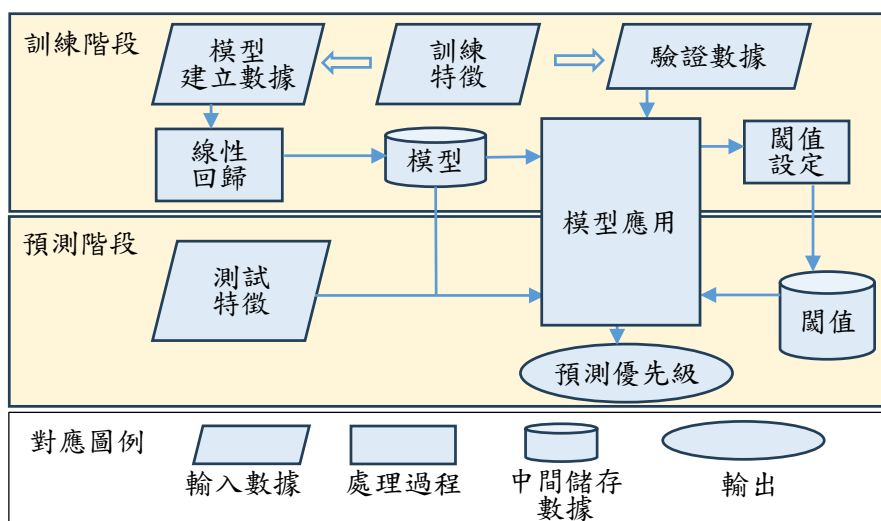


圖四、基於 BERT 的 Triplet Network 結構圖[4]

為改善圖一的步驟 2，Tian 等人曾提出 DRONE 框架[6]以針對錯誤報告進行自動化的優先級判定，此方法運用多因子分析理念(Multi-Factor Analysis)，將表二所列的因子提取為特徵，透過 DRONE 框架內名為 GRAY 的核心組件作為分類引擎(Classification Engine，此引擎內分成兩大部分：線性迴歸與閾值化機制，如圖五所示)，並利用此引擎預測錯誤報告的優先級。

表二、多因子分析理念[6]

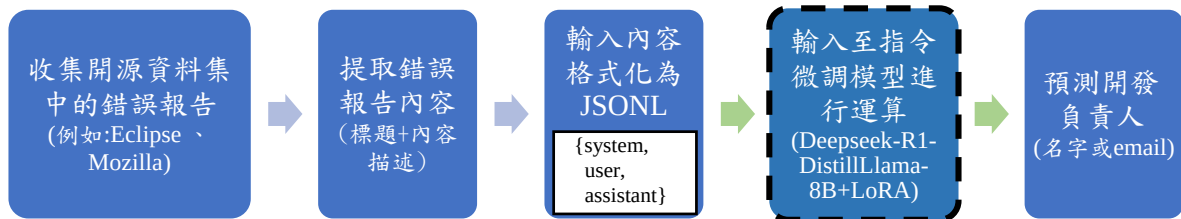
多因子分析	說明
時間因子 (Temporal Factor)	捕捉過去 x 天內以優先級 y 回報的錯誤數量，呈現特定時間範圍內已存在的錯誤報告數量與嚴重程度分布。
文本因子 (Textual Factor)	對錯誤報告的摘要與描述進行資料預處理(分詞、去停用詞、詞幹提取)，分析其內容特徵。
作者因子 (Author Factor)	該使用者過去提交報告的平均優先級、中位數優先級及報告總數。
相關報告因子 (Related-Report Factor)	與當前問題最相似的前幾筆歷史報告之優先級設定。
嚴重性因子 (Severity Factor)	使用者所感知的嚴重性等級。
產品因子 (Product Factor)	特定元件過去被回報的錯誤頻率及歷史優先級分布。



圖五、GRAY 的分類引擎[6]

Bug Triage 是指將被判定為錯誤的 Ticket 指派給最適合的開發人員(如圖一的步驟 3)，傳統作法是依賴資深人員的經驗進行審查與判斷，但這種作法在大型專案中不僅效率低，且難以持續。對此，Kiashemshaki 等人[7]提出基於 JSONL(JSON

Lines，即一行是一個獨立的 JSON)格式的指令微調架構(如圖六所示)，過程是將原始錯誤報告文本中的資料轉換為 JSONL 格式，並輸入到由 LoRA 微調 DeepSeek-R1-Distill-Llama-8B 所得的 LLM 中，最終建議該負責處理該 Ticket 的開發者。該研究以 JSONL 格式設計特定的提示詞模板，將錯誤報告轉換為包含 System(設定角色的 Prompt)、User(問題標題及完整描述的組合)與 Assistant(指定開發人員的電子郵件或姓名)角色的專家對話形式(請參見圖七)，並透過監督式微調(Supervised Fine-Tuning，後續簡稱 SFT)技術訓練模型(如圖八)。也就是說，將通用的 LLM 調整為能夠遵循特定指令並執行特定任務(如自動化錯誤報告分流)的專用模型，使其能精確地學習錯誤與開發者之間的指派邏輯。



圖六、文獻[7]的 LLM 訓練與預測流程圖

```
"system": "You are an expert bug triager.",  
"user": "Title: App crashes when saving.  
nDescription: Steps to reproduce...",  
"assistant": "dev@example.com"
```

圖七、經 LLM 微調的 JSONL 指令範例[7]

```
Below is an issue. Suggest the single best developer to resolve it.  
### Issue:  
{title + "\n\n" + body}  
### Assignee: {gold_assignee}
```

圖八、STL 訓練用的指令模板[7]

3.1.3 自動化錯誤定位與補丁生成

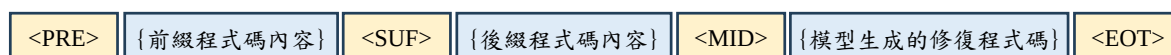
錯誤定位與補丁生成的自動化可輔助圖一的步驟 5，加速 Ticket 所回報錯誤的修復。Rozière 等人[1]指出，傳統 LLM 所支援的上下文通常僅約 4,096 Tokens，故只能容納一個單獨的函式或一個較小的檔案。由此看來，LLM 適用於函式級(Function-Level)的錯誤定位。當錯誤涉及函式的呼叫時，因 LLM 的上下文可能未包含其它函式或檔案的內容，推理能力只被鎖在單一函式內，影響錯誤定位成效。文獻[1]亦指出，Code Llama 系列皆支援高達 100,000 Tokens 的長輸入上下文，是其技術上的一大突破。這項特性解決傳統的 LLM 僅能進行函式級推理的限制，使其具備儲存庫層級(Repository-Level)的推理能力，意味著它能一次讀入整個專案的多個核心檔案。這使得系統能夠進行跨檔案分析，理解不同模組間的邏輯關聯，從而精確定位錯誤的根源(即最可能發生錯誤的檔案、函式或程式碼區塊)。Rozière 等人[1]參考 Liu 等人[8]的做法進行關鍵資訊檢索(Key Retrieval)實驗[1](該實驗的設計概念是參考並類比 Liu 等人[8])，此實驗被用來驗證 Code Llama 系列處理長輸入上下文的能力，故即使在極長的文件中，模型仍能維持高度的準確性來定位最可能隱含錯誤的程式區塊，這支持專題中跨檔案分析錯誤來源並精準定位的設計。

另一方面，在文獻[1]的範例顯示，Code Llama-Instruct 能精準指出 Python 程式碼中的錯誤成因，並提供修正後的函數。但是傳統 LLM 通常採用從左往右的標記預測，它們在生成程式碼時只能參考過去的訊息(前綴)，而無法預知未來的程式碼(後綴)，這種單向性會導致補丁難以與下方的既有的程式碼無縫銜接。所以 Code Llama-Instruct [1]是採用 Fill-In-the-Middle(簡稱 FIM)機制[9]的填充技術，

在訓練時將程式碼序列重新排列為(Prefix, Suffix, Middle)的格式(亦稱為 PSM 格式)。這使得模型在生成位於中間(Middle)的修復內容時，可參考錯誤定位的上下文資訊(前綴與後綴)填充修復的補丁，從而確保生成的補丁能銜接既有程式碼。這項技術突破傳統 LLM 僅能由左至右生成的局限。FIM 機制是根據 Bavarian 等人[9]所提及的哨點標記(Sentinel Tokens，如<PRE>、<SUF>、<MID>與<EOT>，各標記的說明如表三)來標示程式碼邊界(標示方式如圖九說明)，並在<MID>標記後精確預測應填補的修復程式碼，直到產出<EOT>標記為止。

表三、各哨點標記說明[9]

哨點標記	說明
<PRE> (Prefix)	放至錯誤位置之前的原始程式碼最前面。
<SUF> (Suffix)	放至錯誤位置之後的原始程式碼最前面。
<MID> (Middle)	標示中間部分(填充內容)的開始。這是模型接收指令後，準備開始生成補丁內容的信號點。
<EOT> (End of Infilling)	標示填充結束。當模型生成的內容成功與後綴連接時，會預測出此標記，代表修復補丁已完成。



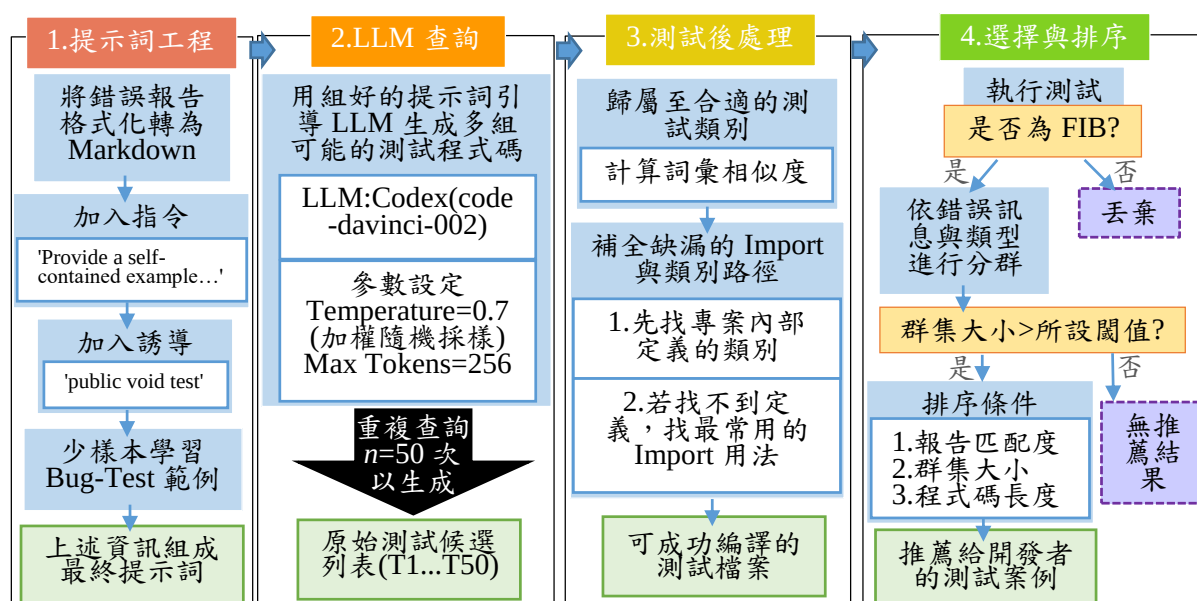
圖九、程式碼邊界格式示意圖(以 PSM 格式為例)[9]

3.2 自動化驗證與測試階段

本階段旨在驗證 LLM 所生成的修復補丁是否有效，並確保修復過程未增加新的錯誤。我們將驗證流程分為「重現錯誤的有效性驗證」與「確保系統穩定性的回歸測試」兩部分。

3.2.1 自動化測試案例生成

為了證明使用者所描述之錯誤確實已被修復，最直接的方法是執行能重現該錯誤的測試。對此問題，Kang 等人[10]提出的 LIBRO 系統展示了 LLM 具備從自然語言錯誤報告中自動生成重現測試案例的能力。如圖十所示，該方法透過提示工程引導 LLM 生成多個候選測試，並經由後處理機制解決程式碼依賴性問題。關鍵在於，該研究確立驗證標準：Fail In Buggy(後續簡稱 FIB)，有效的測試必須在錯誤版本中失敗，並在修復版本中通過。



圖十、LIBRO 系統運作流程圖[10]

3.2.2 回歸測試

除了驗證錯誤是否被修復，軟體維護的另一關鍵是確保新生成的補丁未破壞既有功能。Liu 等人[11]提出的 CLEVEREST 是一種基於 LLM 的回饋導向(Feedback-Directed)、零樣本回歸測試產生技術，專注於探討針對單次程式碼變更的測試生成。該系統的運作流程主要由提示合成器、LLM 模組和執行分析器三個核心組件構成。

首先，提示合成器會建立包含任務描述、提交訊息的資訊及嘗試歷史紀錄(於回歸測試案例未達成預期效果時，會被記錄成歷史紀錄)的提示詞。接著，LLM 模組依據此提示生成測試輸入，並交由執行分析器對修復前後的版本進行比對，利用 RIPR(Reaching, Infecting, Propagating, and Revealing)模型[12]評估測試輸入的有效性，檢測其是否僅覆蓋變更程式碼(對應 Reaching)、導致輸出行為差異(對應 Infecting 與 Propagating)，或是成功觸發錯誤(對應 Revealing)。若測試未能揭露錯誤，分析器會將詳細的執行結果回饋至提示合成器，透過迭代優化過程，引導 LLM 生成更具效力的回歸測試案例。

3.3 提交訊息的生成

Code Llama-Instruct 模型具有零樣本指令遵循(Zero-Shot Instruction Following) [1]，使其能直接理解使用者的自然語言描述之修復意圖，再透過此修復意圖引導模型執行辨識並解釋程式碼中的邏輯錯誤等任務，因而得知程式碼錯誤成因。此模型亦結合 FIM 填充技術執行程式碼摘要生成文檔(如 Documentation Strings，簡稱 Docstrings)的能力，而這項能力已透過 CodeXGLUE³基準測試證明能有效生成如 Docstrings 等技術文件，進而將模型生成的複雜程式碼補丁(程式碼修改描述)轉化為人類可讀的文字描述。綜上所述，該模型不僅展現對於程式碼語義的理解，更具備同時處理程式碼錯誤成因的分析與修改的描述之技術特質。若仿照文獻[1]整合 Code Llama-Instruct 模型與 FIM 填充技術，並搭配適當的提示工程，則有機會自動生成提交程式碼時需要填寫的訊息，細節將於章節(四)說明。

(四) 研究方法及步驟

本專題「基於 LLM 的軟體錯誤追蹤與修復系統」將研究過程劃分為以下五個核心步驟進行：

步驟 1：資料前處理與標準化架構建置

本步驟的目標為解決非結構化 Ticket 難以分析的問題，建立資料正規化的流程。

步驟 1.1：資料集建置

我們將蒐集(1)開源專案中的錯誤報告和提交紀錄，(2)提供真實軟體錯誤案例的資料集，和(3)技術論壇(例如：Stack Overflow⁴)，建立包含錯誤描述、Log 資訊、正確補丁的標準訓練與測試資料庫。

步驟 1.2：非結構化資訊擷取並進行資料正規化

此步驟採用 Code Llama-Instruct，該 LLM 透過 RLHF V5 使其具備零樣本指令遵循能力，以精準分析使用者輸入的自然語言描述，並結合提示工程結構化的 JSON 格式。藉由其強大的長文本推理能力，LLM 可自動提取包含問題類型、受影響模組、作業系統環境與版本號等關鍵欄位。

步驟 2：自動化 Ticket 分類與分流模型開發

本步驟將引入語意分析技術，解決重複回報與人工分派耗時的問題。

步驟 2.1：重複 Ticket 識別

不同於傳統關鍵字比對，本專題擬運用 SBERT 及 Triplet Network，以針對 Ticket 進行向量化比對，透過計算新進 Ticket 與歷史 Tickets 的向量相似

³ <https://microsoft.github.io/CodeXGLUE/>

⁴ <https://stackoverflow.com/>

度，以精準識別重複的 Ticket。若相似度高於設定閾值，系統將自動標記為 Duplicate。

步驟 2.2：優先級分類

針對表一所呈列的五個優先級，我們欲採用 DRONE 框架，運用多因子分析整合六大特徵以預測優先權。分類核心採用 GRAY 引擎，透過線性迴歸捕捉 P1 至 P5 階層間的順序性數值關係，並結合閾值化機制自動校準資料不平衡的問題(即多數 Tickets 集中於某個優先級的情況，例如：多數的 Tickets 都被標註為預設的 P3，而 P1 或 P4 非常稀少)，以避免優先級分布過於集中。

步驟 2.3：自動化分流

參考結合 JSONL 指令微調架構以訓練 LLM(採用 Code Llama-Instruct)，使其可以依 Ticket 的內容，分析並分派給適合的負責人。

步驟 3：自動化錯誤定位和補丁生成與驗證

本步驟為系統核心，旨在利用 LLM 輔助工程師快速找到問題根源以及提供修復補丁。此外，本步驟也將確保生成的補丁具備正確性與安全性，避免引入新的錯誤。

步驟 3.1：錯誤定位

將利用支援長文本的 Code Llama-Instruct 進行儲存庫層級的推理，分析錯誤描述與專案程式碼的關聯。系統將輸出最可能發生錯誤的檔案、函式或程式碼區塊，以縮小工程師的檢視範圍。

步驟 3.2：補丁生成

我們將微調 Code Llama-Instruct，使其具備辨識並解釋邏輯錯誤成因的能力。在分析錯誤後，將會自動生成修正後的函數或程式碼片段(即補丁)。接著，將應用 FIM 填充技術，精準地將補丁放置於正確位置。

步驟 3.3：自動化測試案例生成以重現 Ticket 描述的錯誤

參考 LIBRO 系統的機制，利用提示工程引導 LLM 根據 Ticket 內容生成多個候選測試案例。接著，透過分析這些測試的執行結果與失敗輸出(執行程式碼失敗時產生的所有資訊)的一致性，篩選出高信賴度的測試案例(可重現 Ticket 描述之錯誤)。若該測試案例在套用補丁前執行失敗，而在套用後執行通過，即視為修復有效，以此作為驗證自動化修復有效性的核心指標。

步驟 3.4：回歸測試

為確保修復過程未破壞系統既有的功能運作，在此將導入 CLEVEREST 機制進行驗證。系統首先將任務描述與提交訊息等資訊整合為提示詞，引導 LLM 生成測試輸入，並透過執行分析器在程式碼變更前後的版本上進行對比。接著，依據 RIPR 模型指標評估測試之有效性。若測試未能有效覆蓋變更或揭露差異，系統將自動把執行結果回饋至提示合成器進行迭代優化，直至產出有效具備檢測力的回歸測試案例，以確保修復後的程式碼是正常運作的。

步驟 4：提交訊息生成

將利用 Code Llama-Instruct 具備的零樣本指令遵循分析錯誤成因。另外，針對此 LLM 產出的程式碼補丁，本專題應用 FIM 填充技術執行程式碼摘要與 Docstrings 的生成，將複雜的程式碼異動轉為人類可讀的修改描述。最後，利用提示工程，整合程式碼的錯誤成因與修改描述，產出提交訊息的草稿。

步驟 5：系統測試與優化

步驟 5.1：功能整合與測試

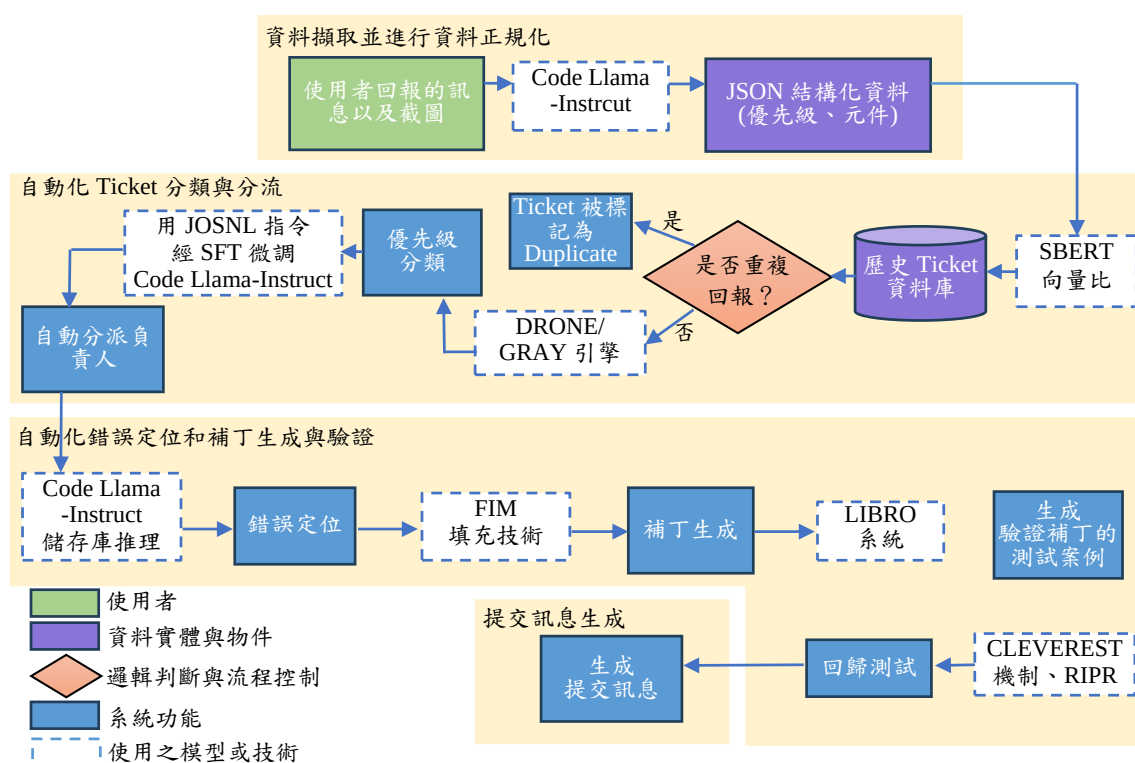
此步驟將前述各項功能以由下而上的方式整合為完整應用，並持續進行整合測試。

步驟 5.2：成效評估

我們將以表四所列資料集檢驗欲實作的各項系統功能，並依檢驗結果撰寫專題成果報告。

表四、功能對應資料集

功能	使用的資料集
資料結構化成 JSON 欄位	DeepJSONEval ⁵
Ticket 重複檢測準確度	TAWOS ⁶
Ticket 分類與分流	
錯誤定位精準度	Defects4J ⁷
補丁修復成功率(即通過測試案例之比例)	
提交訊息生成的完整性	CommitBench ⁸



圖十一、系統功能架構

(五) 預期結果

本專題預計開發一套「基於 LLM 的軟體錯誤追蹤與修復系統」(其系統功能架構如圖十一所示)，並預期達成以下具體成果：

1. 標準化與自動化的資料處理：系統能運用 Code Llama-Instruct 的零樣本指令遵循，將使用者模糊的錯誤描述自動轉化為標準 JSON 格式，大幅降低人工整理資訊的時間成本。
2. Tickets 分類與分流：透過 SBERT 與分類模型，預期能精準識別重複 Ticket，同時藉由整合多因子分析與 GRAY 分類引擎，預期能將優先級視為順序性數值進行精準預測，並利用閾值化技術有效校準資料不平衡的問題。此外，系統以能為 Tickets 自動分派負責人，解決傳統錯誤追蹤系統依賴人工經驗的瓶頸。

⁵ <https://github.com/GTS-AI-Infra-Lab-SotaS/DeepJSONEval>

⁶ <https://github.com/SOLAR-group/TAWOS>

⁷ <https://github.com/program-repair/defects4j-dissection>

⁸ <https://github.com/Maxscha/commitbench>

3. 自動化的輔助修復功能：系統能應用 FIM 填充技術提供準確且能與既有程式碼無縫銜接的補丁，並自動生成具有程式碼修改原因與範圍的提交訊息草稿，協助工程師快速進入審核階段。
4. 自動化測試驗證能力：系統除能自動生成測試案例以驗證錯誤的修復狀況外，亦能透過回歸測試機制確保生成的補丁未破壞系統既有功能的運作，以提升修復的安全性。
5. 提升整體開發效率：透過整合上述功能，預期能縮短從問題回報到錯誤修復的平均時間(Mean Time to Repair，簡稱 MTTR)，以提升軟體維護的品質與效率。

(六) 需要指導教授指導內容

此專題的進度規劃如表五，在此研究進行過程，預計每週定期向指導教授報告進度，並請指導教授針對以下三項給予建議：

1. 系統設計與技術選擇指導：在 Ticket 分析流程、錯誤定位與補丁生成等核心模組的設計上，協助評估不同技術方案的適切性，並提供模型選用、Prompt 設計與系統整合方面的建議。
2. 實驗設計與評估方法建議：指導實驗流程的規劃方式，包括測試資料集選擇、評估指標設定(如分類準確度、錯誤定位精準度與修復可行性)，以及結果分析與比較方式。
3. 研究成果整理與論文撰寫指導：協助釐清研究成果的呈現重點，並在計畫書與成果報告撰寫上提供架構與表達方式的建議，使內容能清楚展現研究貢獻與實務價值。

表五、專題排程計畫表

行程規劃/時間	115 年 7 月	115 年 8 月	115 年 9 月	115 年 10 月	115 年 11 月	115 年 12 月	116 年 1 月	116 年 2 月
步驟 1. 資料前處理與標準化架構建置								
步驟 2. 自動化 Ticket 分類與分流模型開發								
步驟 3. 自動化錯誤定位和補丁生成與驗證								
步驟 4. 提交訊息生成								
步驟 5. 系統測試與優化								
撰寫成果報告								

(七) 参考文献

- [1] B. Rozière, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, R. Sauvestre, T. Remez, J. Rapin, A. Kozhevnikov, I. Evtimov, J. Bitton, M. Bhatt, C. C. Ferrer, A. Grattafiori, W. Xiong, A. Défossez, J. Copet, F. Azhar, H. Touvron, L. Martin, N. Usunier, T. Scialom, and G. Synnaeve, “Code Llama: Open Foundation Models for Code,” preprint arXiv:2308.12950, 2024.
- [2] G. Salton and M. J. McGill, *Introduction to Modern Information Retrieval*, McGraw-Hill, 1983.
- [3] D. M. Blei, A. Y. Ng, and M. I. Jordan, “Latent Dirichlet Allocation,” *Journal of Machine Learning Research*, Vol. 3, pp. 993-1022, 2003.
- [4] H. Isotani, H. Washizaki, Y. Fukazawa, T. Nomoto, S. Ouji, and S. Saito, “Sentence Embedding and Fine-Tuning to Automatically Identify Duplicate Bugs,” *Frontiers in Computer Science*, Vol. 4, Article No. 1032452, 2023.
- [5] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks,” *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, pp. 3982-3992, 2019.
- [6] Y. Tian, D. Lo, X. Xia, and C. Sun, “Automated Prediction of Bug Report Priority Using Multi-Factor Analysis,” *Empirical Software Engineering*, Vol. 20, No. 5, pp. 1354-1383, 2015.
- [7] K. Kiashemshaki, A. Khosravani, A. Hosseinpour, and A. Akhavan, “Automated Bug Triaging Using Instruction-Tuned Large Language Models,” preprint arXiv:2508.21156, 2025.
- [8] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, “Lost in the Middle: How Language Models Use Long Contexts,” *Transactions of the Association for Computational Linguistics*, Vol. 12, pp. 157-173, 2024.
- [9] M. Bavarian, H. Jun, N. Tezak, J. Schulman, C. McLeavey, J. Tworek, and M. Chen, “Efficient Training of Language Models to Fill in the Middle,” preprint arXiv:2207.14255, 2022.
- [10] S. Kang, J. Yoon, and S. Yoo, “Large Language Models Are Few-Shot Testers: Exploring LLM-Based General Bug Reproduction,” *Proceedings of the 45th International Conference on Software Engineering*, pp. 2312-2323, 2023.
- [11] J. Liu, S. Lee, E. Losiouk, and M. Böhme, “Can LLM Generate Regression Tests for Software Commits?” preprint arXiv:2501.11086, 2025.
- [12] N. Li, and J. Offutt, “Test Oracle Strategies for Model-Based Testing,” *IEEE Transactions on Software Engineering*, Vol. 43, pp. 372-395, 2017.